structure:

- We could sort the input and take the last `k` elements. This naïve approach requires `O(n*log(n))` comparisons and swaps and, depending on the algorithm, might require additional memory.
- We could find the largest element from the set and move it to the end of the array, then look at the remaining `n-1` elements and find the second last and move it to position `n-2`, etc... Basically, this algorithm runs the inner cycle of `Selection Sort` algorithm `k` times, requiring `O(k)` swaps and `O(n*k)` comparisons. No additional memory would be needed.

In this section we will see that, by using a heap, we can achieve our goal using `O(n+k*log(k))` comparisons and swaps, and `O(k)` extra memory. This is a game changing improvement, if `k` is much smaller than `n`. In a typical situation `n` could be of the order of millions or billions of elements, and `k` between a hundred and a few thousands.

Moreover, by using an auxiliary heap the algorithm can naturally be adapted to work on dynamic streams of data, and also to allow consuming elements from the heap.

2.7.1 The right data structure...

When your problem involves finding a subset of largest/smallest elements, priority queues seems a natural solution.